

AI로 만든 SessionCast, 그런데 왜 Spring을 골랐을까

핵심 원리부터 AI + Spring 실전까지 — 토론

노성현

SESSIONCAST.IO · GITHUB.COM/SESSIONCAST

2026 SPRING CAMP

01 / 42

한 줄로 시작합니다

PC에서 돌고 있는 AI 작업들, 밖에서 나가서 할 순 없을까?

— 이 한 줄이 SessionCast의 시작

TODAY'S JOURNEY

오늘의 여정 — 네 막.

1막

왜 만들었나

밤샘 에피소드 + 4단계 문제 → SessionCast

5 MIN

2막

어떻게 돌아가나

tmux 핵심 원리 + Relay 아키텍처 + 라이브 데모

10 MIN

3막

AI + Spring으로 빠르게 만든다면?

실제 AI 협업 과정 · Context Engineering 실전

15 MIN

4막

왜 Spring이었나

4가지 이유 — 체험 후라 짤아도 설득력

7 MIN

03 / 42

ACT ONE

ACT ONE

01

왜 만들었나

에이전트 오케스트레이션, 그리고 그날 밤 — 자리를 뜨면 끝이었다.

5 MIN

이미 이렇게 쓰고 있었다.

여러 Claude Code 에이전트를 동시에 띄우고, tmux send-keys로 서로 대화하며 협업하는 시스템을 굴리고 있었다.



PM Agent가 Dev·QA에게 **send-keys**로 지시



QA가 FAIL → 오케스트레이터가 Dev에 재시도 요청

PM이 흐름을 관리하고 — Dev·QA·Doc·Deploy가 **send-keys**로 서로 대화하며 끝까지 진행. 사람은 시작만 누른다.

그런데 — 자리를 뜨면 끝이었다.

밤새 AI랑 작업하고 새벽 4시에 끝. 오전 고객사 미팅 — 도착해서야 알았다.

설치 가이드를 안 만들고 왔다.

AI에게 "이거 만들어줘" 한 마디면 끝났을 일. 그런데 — AI는 내 노트북에서 돌고 있었다.

"내 노트북에서 돌고 있는 AI를,
어디서든 부를 수 있어야 한다."

지금까지는 — 4단계, 23개 작업.

모바일에서 한 글자 치려면, 이 4단계를 다 끝낸 다음에야 가능했다.

01VPN TAILSCALE 같은 메시 VPN ☐ 계정 생성 · 인증 ☐ 노트북 · 폰 양쪽 설치 ☐ 방화벽 예외 · ACL 확인	02모바일 터미널 TERMIUS / TERMUX ☐ 앱 설치 · 구독 결제 ☐ 키 import · 동기화 ☐ 단축키 · 폰트 설정	03SSH 키 발급 → 배포 → 권한 ☐ ssh-keygen · 폰 전송 ☐ authorized_keys 등록 ☐ chmod · known_hosts	04연결 유지 끊김 대응 · 재접속 ☐ sleep 막기 ☐ mosh · keepalive ☐ 끊김 시 세션 복구
6개 작업	5개 작업	6개 작업	6개 작업

"4단계"라고 부르지만 — 실제로는 23개의 작은 작업. 하나라도 빼놓으면, 한 글자도 못 친다. 평균 셋업 1-2시간.

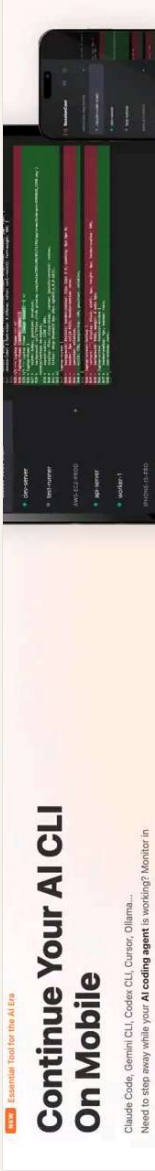
통째로 없앤다.

BEFORE · 4 단계

~~VPN 설치 인증~~
~~모바일 터미널 앱 구독~~
~~SSH 키 발급 등록 권한~~
~~연결 유지 mosh keepalive~~

AFTER · 0 단계

sessioncast 한 줄.
+ Google 로그인.
— 끝.



↑ 실제 설치 → 로그인 → Agent 시작 → 세션 자동 발견까지 (curl 한 줄 + Google 로그인)

이제 —

SessionCast.

한 줄 명령 + 한 번 로그인 — PC의 AI를 어디서든 부른다.

'무엇'인지는 알았다. 이제 — '어떻게' 돌아가는지.

ACT TWO

ACT TWO

02

어떻게 돌아가나

tmux의 마법 + Relay 아키텍처 + 라이브 데모.

10 MIN

기준 원격 접속과는 — 근본적으로 다른 방식.

기준 · VPN + SSH

터미널에 직접 접속한다.

네트워크 터널을 뚫어 — 내가 그 터미널 '안에' 들어간다.
그래서 VPN · SSH 키 · 연결 유지가 전부 필요했다.

SESSIONCAST · CAPTURE + RELAY

화면을 읽어서 중계 한다.

안에 들어가는 게 아니라 — capture-pane 으로 화면을 읽고, send-keys 로 키를 넣는다.

VPN도, SSH도, 터미널 앱도 필요 없다.

이 차이가 핵심 — 이제 capture-pane 과 send-keys 가 어떻게 동작하는지 본다.

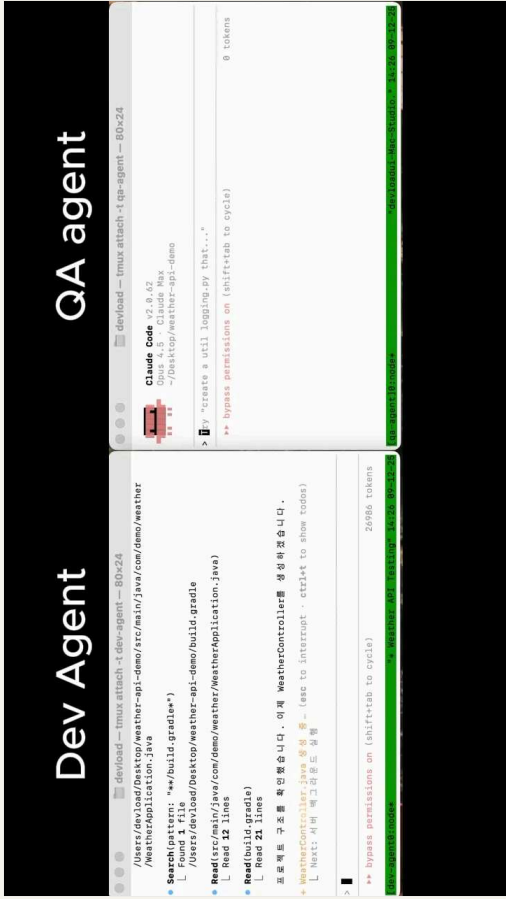
tmux capture-pane — 화면을 읽는다

PRINCIPLE 01

새 세션을 만드는 게 아니라, 이미 돌고 있는 tmux 세션의 화면 버퍼를 텍스트로 뽑아낸다. Claude Code가 도는 세션을 건드리지 않고, 옆에서 화면만 읽는다.

```
# Agent가 50ms마다 실행하는 핵심 명령
tmux capture-pane -t <session> -p -e -N
-p stdout 출력 -e ANSI 색상 -N 공백 보존
```

캡처한 화면 → WebSocket으로 Relay → Viewer의 xterm.js가 렌더링.
초당 20프레임.



↑ Dev Agent가 코드를 짜는 tmux 세션. capture-pane 이 이 화면을 텍스트로 뽑는다.

PRINCIPLE 02

tmux send-keys — 입력을 보낸다

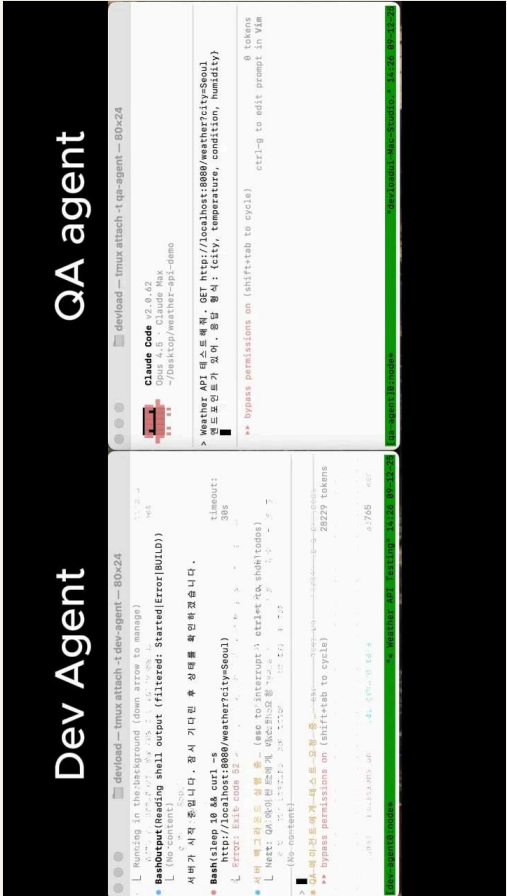
보는 것만이 아니라 — 키 입력을 세션 안에 주입할 수 있다.

```
# 리더샵 텍스트 - 문자 그대로
tmux send-keys -t session -l "y"

# 특수 키
tmux send-keys -t session Enter
tmux send-keys -t session C-c
```



12 ms



↑ 같은 원리로 폰 → Relay → Agent → send-keys → 에이전트 간 다리가, 사람-AI의 다리.

화면이 흐르는 방식.

조 건	통 작	이 유
화면 변경 (최근 2초)	50ms마다 capture-pane	초당 20프레임 — 실시간
비활성	200ms마다 capture-pane	AI 생각 중 — 배터리 절약
무조건	10초마다 강제 전송	Viewer가 멈춰 보이지 않게
데이터 ≥ 512 byte	gzip → base64 → screenGz	모바일 대역폭 절약

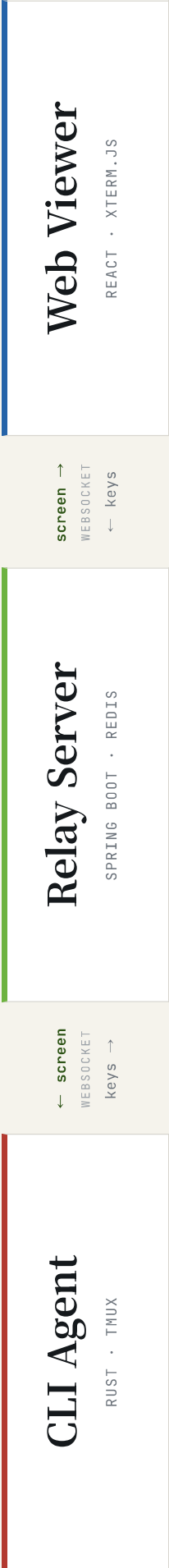
새 Viewer 접속

캐시된 마지막 화면 즉시 전달

빈 화면에서 시작 안 함

Relay는 Viewer 접속 시 `startStream`, 전원 퇴장 시 `stopStream` 을 Agent에 보낸다 — 아무도 안 볼 땐 캡처하지 않는 배터리 최적화.

이걸 엮으면 — 이 구조가 된다.



capture-pane으로 화면을 읽고, send-keys로 입력을 넣는다 — 이 두 원리를 인터넷으로 연결하면 이 구조가 된다.

VPN이나 SSH 터널이 없어도, capture한 텍스트를 WebSocket으로 중계 하면 어디서든 같은 터미널을 본다.

Relay — 양방향 중계 서버. Spring Boot.

RelayWebsocketHandler extends TextWebSocketHandler

메시지

방향

동작

register (host)

Agent → Relay

세션 등록, 토큰 검증

register (viewer)

Viewer → Relay

JWT 검증, 캐시된 화면 즉시 전송

screen

Agent → Viewer

화면 브로드캐스트

keys

Viewer → Agent

키 입력 전달 (소유자 검증)

startStream

Relay → Agent

Viewer 접속 시 스트리밍 시작

stopStream

Relay → Agent

전원 퇴장 시 중지

요모

30+ 메시지 타입 — 세션 생명주기, 멀티 pane, 녹화, API 라우팅까지.

Relay가 여러 대일 때 — Redis Pub/Sub.

Agent가 Relay-1에, Viewer가 Relay-2에 붙으면?



LOCAL SESSION REGISTRY

단일 인스턴스 — 개발 / 셀프호스팅.

SPRING

SessionRegistry

인터페이스 분리가 이 전환을 코드 한 줄 차이로 만든다 — 4막에서 다시 짚을 포인트.

REDIS SESSION REGISTRY

멀티 인스턴스 — SaaS. 세션을 Redis에 저장, Pub/Sub로 전달.

Agent — 노트북에서 도는 CLI. Rust.

컴포넌트 / 주기

연결

AgentRunner

5초마다 tmux 세션 스캔 → SessionHandler 생성

SessionHandler

세션당 WebSocket 1개

50ms / 200ms

capture-pane (화면)

15초

메타 정보 — cwd, git branch, worktree

수신

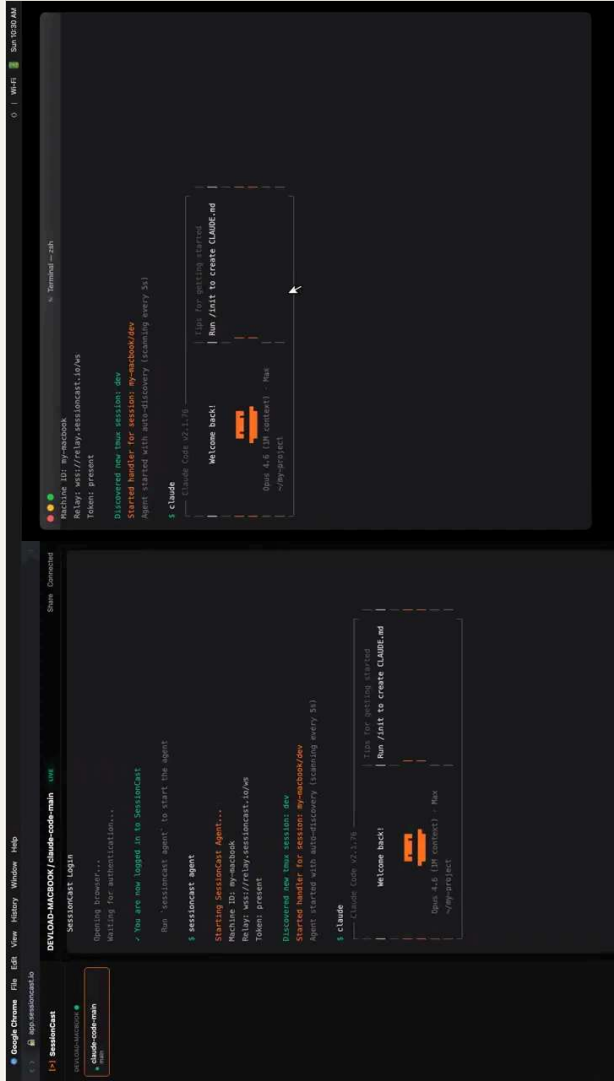
keys · resize · kill · file 요청

왜 RUST?

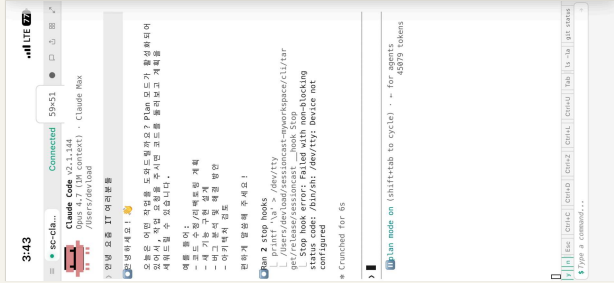
노트북 위 상시 실행. CPU/메모리 최소화. 50ms 캡처 루프에서 GC 멈춤은 치명적 — Relay는 Spring, Agent는 Rust. 이 선택도 4막에서.

 LIVE DEMO

노트북에서 돌던 세션이, 폰에 그대로.



↑ 좌: 웹 콘솔 · 우: 노트북 터미널 — 같은 Claude Code 세션이 실시간 동기화



폰 — 같은 세션, 입력도 가능.
(라이브 실패 시 녹화 백업)

ACT THREE

ACT THREE

AI + Spring으로 빠르게 만든다면?

*SessionCast*을 실제로 만든 과정 — *Context Engineering*
실전.

15 MIN · 발표의 핵심

20 / 42

AI로 만든 — 네 개의 Spring 서비스.

Relay Agent ↔ Viewer 실시간 중계 — 이게 없으면 아무것도 안 돌아간다.	Auth 내 세션에 나만 접근 — 최소 보안.
Platform 사용자 · 토큰 관리 (Relay 밖으로).	Gateway 앞단 라우팅 · Rate Limit.

MVP는 **Relay · Auth 2개**로 시작 → 지금 **4개**. 1인 + AI로 운영 중 — 모듈이 늘어나니까, AI한테 시킬 때 새로운 문제가 생겼다.

그래서 — AI한테 이렇게 시켰다.

"Spring Boot로 WebSocket Relay 서버를 만들어줘.

- Agent(호스트)가 접속하면 세션을 등록하고, Viewer가 접속하면 같은 세션에 연결해줘.
- Agent가 보낸 screen 메시지는 Viewer에 브로드캐스트, Viewer가 보낸 keys 메시지는 Agent에 전달.
- 인종은 Agent는 토큰, Viewer는 JWT.
- 세션 저장은 일단 인메모리, 나중에 Redis로 바꿀 수 있게."

웹 구축적으로 시켰다. MVP는 잘 나왔다.

그리고 여기에 하나씩 붙이기 시작했다 — 멀티 인스턴스, 녹화, 토큰 관리, OAuth2...

기능이 붙을수록 — AI는 길을 잃는다.

모듈이 늘어 난 뒤

- ✗ "멀티 인스턴스 추가" → 기존 **SessionRegistry** 인터페이스를 무시하고 새 클래스를 만들
- ✗ "토큰 검증 강화" → 인증 코드가 *Relay·Auth·Platform* 세 곳에 흩어져 들어감
- ✗ "녹화 기능 추가" → 잘 돌아가던 *WebSocket* 핸들러를 **통째로 리팩터링**하려 함

MVP 때 잘 시킨 그 프롬프트가 — 모듈이 늘어나니까 더 이상 안 통한다.

프롬프트의 문제가 아니었다. **컨텍스트의 문제**였다.



CONTEXT ENGINEERING

+1 for context engineering over prompt engineering. 컨텍스트 엔지니어링은 — 다음 단계에 필요한 정확한 정보로 컨텍스트 창을 채우는 섬세한 기술이다.

Andrej Karpathy 전 OPENAI / TESLA AI · X · 2025.06

한 줄의 마법 프롬프트가 아니라 — AI가 일 잘하는 환경을 꼼꼼히 만들어주는 일.
이걸 SessionCast에서 4가지 방법으로 풀었다.

— KNOWHOW 01

서비스별 *CLAUDE.md* — AI에게 모듈의 지도를.

sessioncast/

- └─ **CLAUDE.md** ← 전체 규칙
- └─ server/**CLAUDE.md** ← Relay 전용
- └─ platform/**CLAUDE.md** ← Platform
- └─ auth/**CLAUDE.md** ← Auth
- └─ gateway/**CLAUDE.md** ← Gateway

Claude Code는 그 디렉토리 파일을 만질 때 자동으로 로드한다.
"Relay에 새 메시지 타입 추가" 한 마디에 — 위치·패턴·관련 클래스를 AI가 안다.

SPRING

멀티 프로젝트 구조가 이 패턴을 자연스럽게 만든다.

Relay의 CLAUDE.md — 실제 내용.

```
# Relay Server (sessioncast/server)

## 역할
WebSocket 브로커. Agent와 Viewer 사이 메시지 중계.

## 핵심 클래스
RelayWebSocketHandler · SessionRegistry · SessionInfo
(Local / Redis 두 구현)

## 메시지 추가 시
1. MessageType enum에 타입 추가
2. handleTextMessage에 분기 추가
3. 필요시 SessionInfo에 상태 필드 추가

## 금지 사항
- Platform/Auth 코드 직접 수정 금지
- SessionRegistry 인터페이스 시그니처 변경 금지
```

— KNOWHOW 02

큰 수정 전에 — 계획부터.

1 단계 - 계획만

"Redis Pub/Sub로 메시지 추가하려고 해.
바로 코드 짜지 말고:
- 어느 파일 수정/생성?
- 영향받을 다른 파일은?
- 인터페이스 변경 필요한가?"

2 단계 - 검토 후

"좋아, 그 계획대로.
단, RelayWebSocketHandler는
건드리지 마."

잘못된 방향이면 5분 만에 돌이킨다.

SPRING

인터페이스 · 어노테이션 · DI가 AI의 계획을 '이 인터페이스 구현, 이 빈 정의'로 구체화한다.

실전 — "Relay에 세션 녹화 추가해줘"

AI의 계획

- 1. RecordingService 인터페이스 정의
- 2. S3RecordingService 구현 (@Service, @ConditionalOnProperty)
- 3. 핸들러에 start/stopRecording 추가
- 4. SessionInfo 에 recordingStatus
- 5. screen 버퍼링 → S3 업로드

내 카드

"SessionRegistry는 건드리지 마. 녹화 상태는 SessionInfo에만."

결과 — 2시간 만에 녹화 기능 완성.

인터페이스 분리 + DI 덕에 기존 코드 영향 최소화.

— KNOWHOW 03

수정 후 — 영향 범위 보고.

"수정 끝" 말고 — "어디까지 영향받을 수 있는가" 보고서를 받는다.

```
[수정한 파일]
RecordingService.java           → 신규 인터페이스
S3RecordingService.java        → 신규 구현체
RelayWebSocketHandler.java     → start/stop 분기 추가

[영향받을 수 있는 파일]
SessionInfo.java              → recordingStatus 필드
RelayWebSocketHandlerTest     → 테스트 추가 필요
platform 모듈                  → 영향 없음 (확인)
```

SPRING

빈 의존성 그래프 + 타입 시스템이 영향 추적을 정확하게 만든다.

— KNOWHOW 04

'하지 마' 규칙 — 3중 안전망.

AI는 좋은 의도로 영역을 넘는다. "이거 해"만으론 부족하다.

1 모듈 CLAUDE.md

"WebSocket 핸들러 직접 수정 X. SessionManager 통해서만."

2 작업 프로토포트

"JWT 검증 로직은 건드리지 마."

3 settings.json · permissions.deny

빌드 결과물 · 자동 생성 파일 차단

SPRING

모듈 경계 + 빈 스코프가 가드를 명확하게 — "Relay 모듈만 건드려" = Spring이 이미 그어둔 경계.

“

새 컴포넌트를 만들 때 Claude는
 기존 컴포넌트를 먼저 본다. 임포트, 주변 컨텍스트를
 보고 — 가장 그 코드베이스다운 방식 으로 변경한다.

Anthropic CLAUDE CODE BEST PRACTICES · 공식 문서

AI를 만든 회사가 직접 말한다 — "우리는 코드베이스의 관례를 따른다."
 → 그 관례가 일관된 도구가 유리하다. Spring의 어노테이션 + 컨벤션.

4가지 노하우 = Context Engineering.

노하우

컨텍스트 관점

SPRING이 틀는 부분

서비스별 CLAUDE.md

모듈 컨텍스트를 미리 알아줌

멀티 프로젝트 구조

계획 먼저

코드 짜기 전 맞는지 검증

인터페이스 · DI

영향 범위 보고

실행 후 컨텍스트 확장

빈 의존성 그래프

'하지 마' 규칙

컨텍스트의 경계 명시

모듈 경계 · 빈 스코프

Spring이 컨텍스트 엔지니어링의 어휘가 되어준다.

"바이브 코딩은 코드를 신경 안 쓴다. 보조 코딩은 코드·복잡도·테스트까지 신경 쓴다. AI 지니의 마법 같지 않은 면은 — 취향(taste)이 없다는 것."

Kent Beck XP · TDD 창시자 · 2025.06

"Devin은 모호한 프로젝트를 시니어처럼 처리 못한다. 엔지니어가 미리 범위를 잡아줘야 한다."

Cognition DEVIN AI 팀 · 2025.11

AI가 코딩을 가져갔다. 설제는 가져가지 못했다. 그래서 *Context Engineering*.

ACT FOUR

ACT FOUR

그래서, 왜 Spring을 골랐을까

Java · Kotlin + Spring Boot — 솔직한 정리.

7 MIN

— 먼저, 솔직하게 분리하자

언어의 힘 — 정적 타입이 AI 환각을 잡는다.

이건 Spring의 힘이 아니라 Java / Kotlin이라는 언어의 힘이다.

핵심

AI가 없는 메소드 호출 · 잘못된 타입 반환 → 컴파일 타임에 차단. Python/JS였다면 런타임까지, 혹은 사용자가 겪을 때까지 몰랐을 일.

"AI 에이전트가 TypeScript로 훨씬 잘해서, Python 쓰던 자리를 TypeScript로 바꾸고 있다."

— Armin Ronacher · Flask 창시자 · 2025.10

AI 시대에 '타입이 있는 언어'가 유리하다는 흐름은 진영을 가리지 않는다 — Java/Kotlin은 처음부터 그 자리에 있었다.

프레임워크의 힘 — *Spring Boot*가 준 것.

타입이 있는 언어는 여러 있다. 그 위에 *Spring Boot*를 올린 이유는 두 가지였다.

01 · 3막의 모듈 구조와 공함

멀티 프로젝트 · 인터페이스 + DI · 빈 스키프.

*Spring*이 모듈 경계를 코드로 강제 해줬기에 Context Engineering이 먹혔다. CLAUDE.md에 "이 모듈만 건드려"를 쓸 수 있던 건 *Spring*이 이미 그 경계를 갖고 있어서.

02 · 운영 도구 기반 탑재

Actuator 서비스 · Redis 상태 · **Micrometer** 연결 수 · 지연 · **Profile** 환경 분리. 설정 없이 기본으로 된다는 게 차이 — 1인이 AI 코드를 production에 서 보는 눈.

같은 사실, 시대만 바뀌었다.

과거 - 단점

현재 - 장점

~~타입 선언어 보일러플레이트~~

타입이 AI 환각을 컴파일러가 잡아줌

~~모듈 분리? 과한 설계~~

AI에게 경계를 알려주는 유일한 방법

~~Actuator? 과한 엔터프라이즈 도구~~

1인 + AI 운영에서 유일한 눈

엔터프라이즈가 수십 년 쌓아온 자산이 — AI 시대에 다른 의미로 돌아온다.

그래도, 한계는 있다.

AGENT는 원래 SPRING이었다 → RUST로 전환

노트북 설치를 쉽게 하려면 JVM 없이 단일 바이너리가 필요했다. 그런데 — Spring으로 잘 구조화한 설계를 Rust로 옮기는 건 AI한테 시키니 놀랍도록 쉬웠다. 인터페이스 경계 · 메시지 타입 · 세션 라이프사이클이 이미 명확했으니까. Spring이 한계라서 뻘 게 아니라, 잘 만든 설계가 언어를 넘어 재사용됐다는 이야기.

프로젝트는 React

xterm.js 같은 터미널 UI는 React 생태계가 압도적

유일한 정답 아님

서버 사이드 비즈니스 로직 + AI 협업에서 특히 강했다 — 거기서 만든 설계는 다른 언어로 옮길 때도 자산

IN CLOSING

네 막, 한 줄로.

1막

PC의 AI를 밖에서도. 그래서 SessionCast.

2막

tmux capture-pane + send-keys → *Relay(Spring Boot)* → 브라우저. 이제 핵심 원리.

3막

그냥 던지면 안 된다. Context Engineering + Spring 구조 = 일관된 코드.

4막

프레임워크·타입·컨벤션·운영 — 4개 층의 일관성. 과거의 단점이 현재의 강점으로.

한 줄로 —

AI가 코딩을 가져갔다. 하지만 설계는 가져가지 못했다.

Spring 개발자는 설계 자산을 갖고 있다 — OOP 사고, 디자인 패턴, 추상화 어휘, 운영 경험. 그 자산이 Context Engineering의 어휘가 된다.

AI가 코드를 쓰는 시대 —

설계자의 도구로,
그리고 **일관성의 도구**로 —
Spring을 다시 보게 된다.

Q&A

동의 안 되는 부분 있으면 그냥 던져주세요 — 토론 트랙입니다.

SPEAKER

노성현

sessioncast.io · github.com/sessioncast

TALK

AI로 만든 SessionCast,
그런데 왜 Spring을 골랐을까

42 / 42